

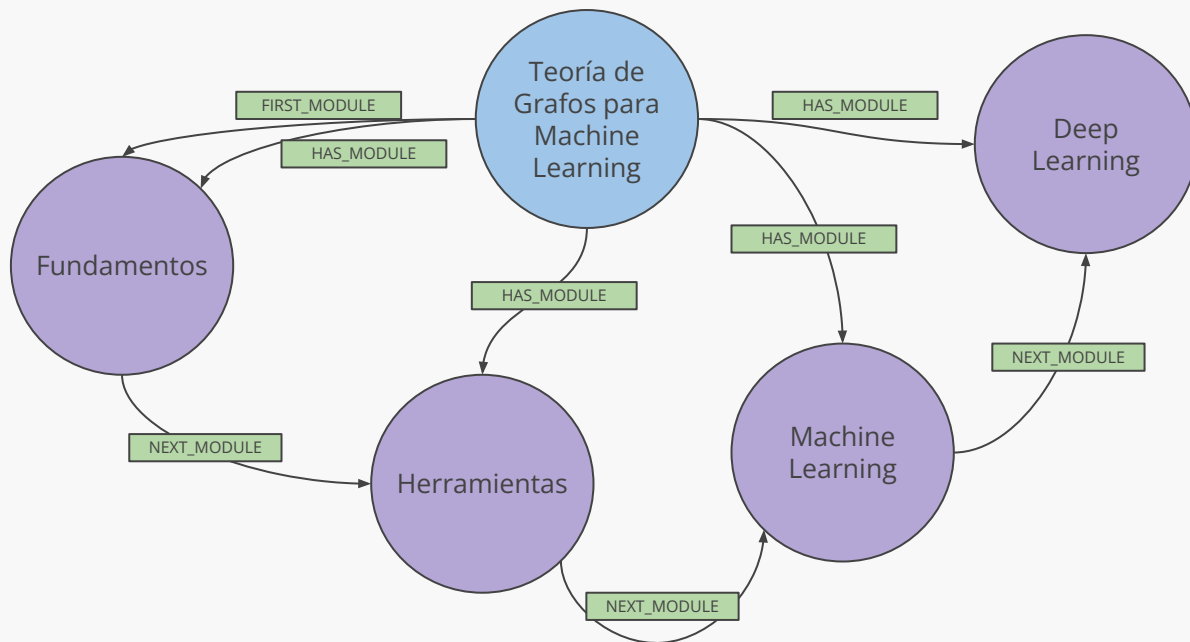
Clase 4.1

Feature engineering

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

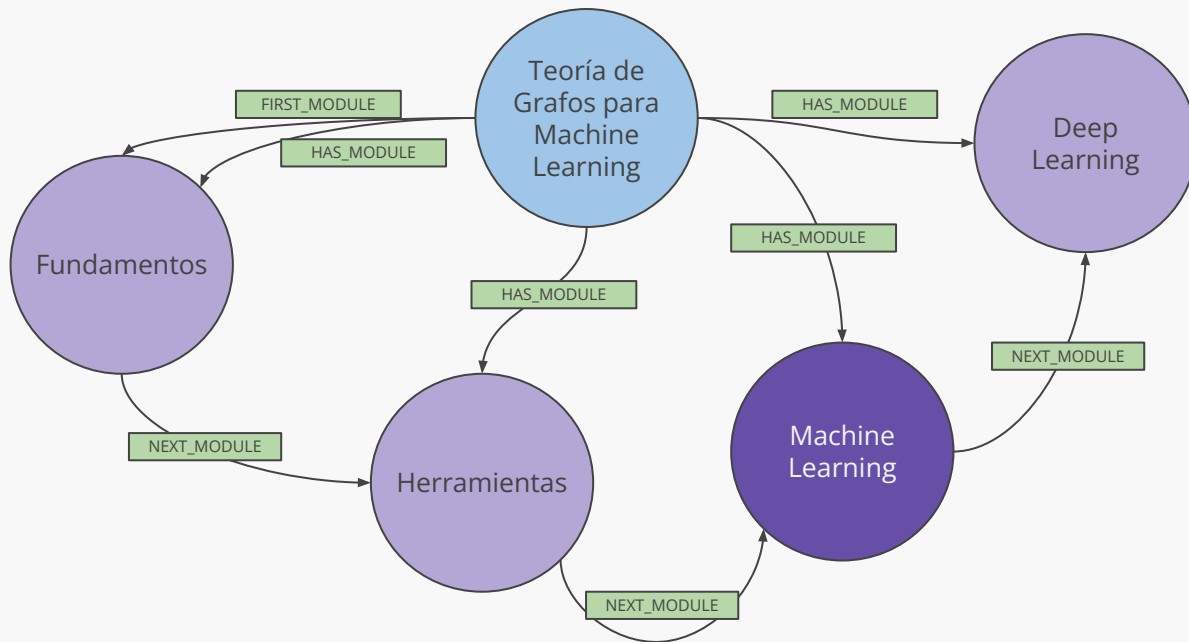


Contenidos del curso

3. Machine Learning

A partir de analítica de grafos, **construir features** que podamos usar en modelos de ML...

... con el objetivo de entender cómo se aplican conceptos de ML sobre grafos



Menti*#Hablemos sobre la EP1*

¿Qué les parece la Evaluación Práctica 1?

Dificultad percibida (de fácil a difícil)

5.3

Interesante/entretenida (de aburrida a entretenida)

5.1

Nota esperada (de 1 a 7)

5.3

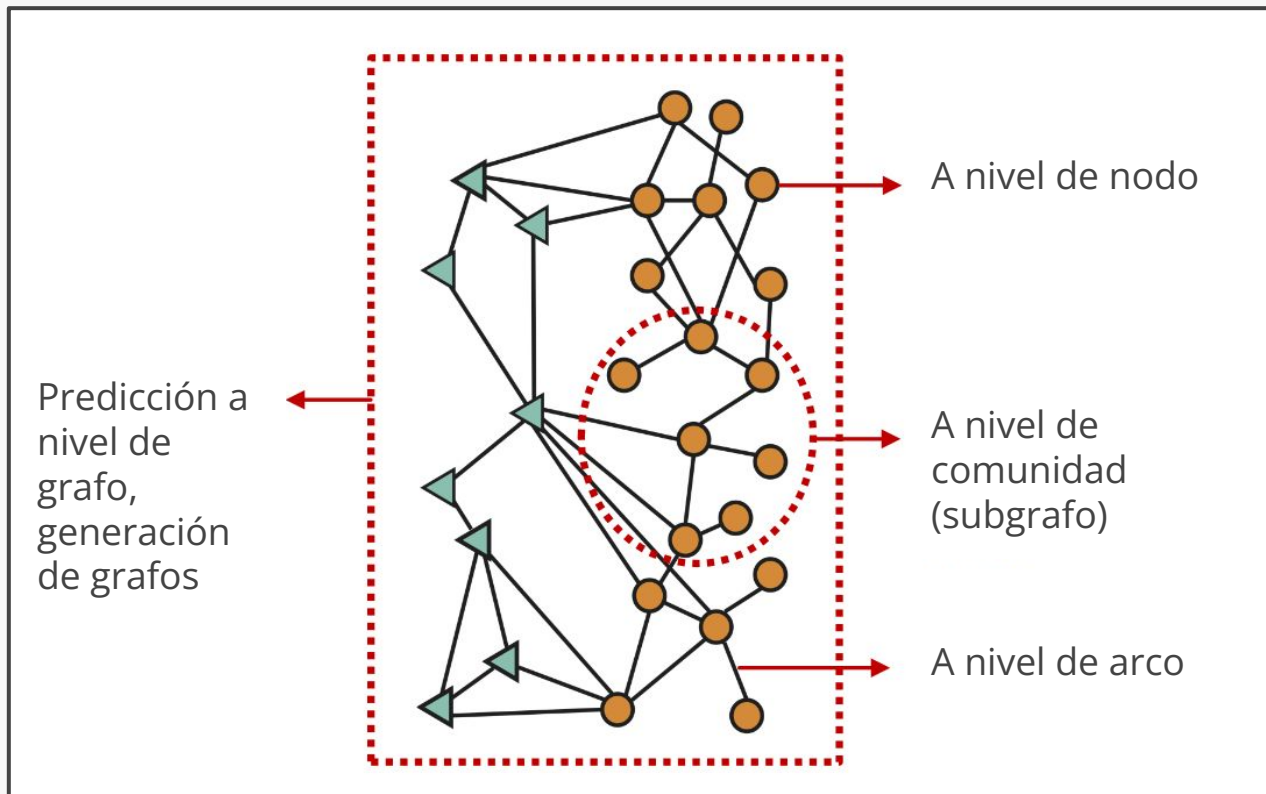
Poco

Alto

¡Gracias por sus respuestas!

Vamos con la clase de hoy

Tipos de tareas

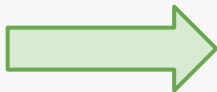


El problema es su estructura “dinámica”

Audio



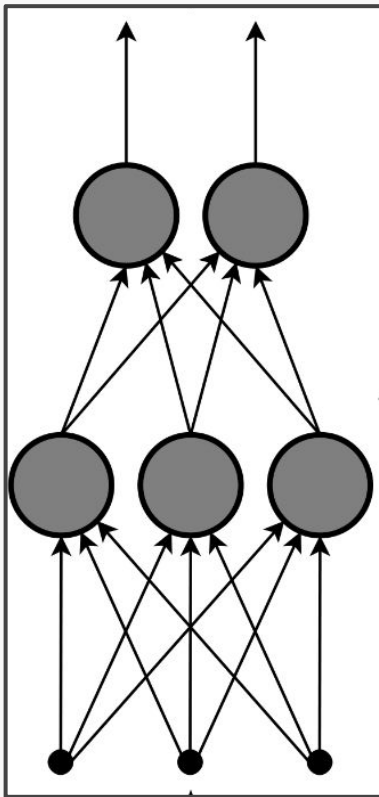
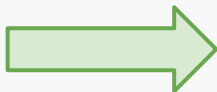
WaveNet, Wav2Vec, ...



Texto



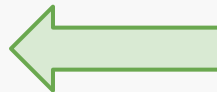
Word embeddings,
Seq2Seq, Transformers, ...



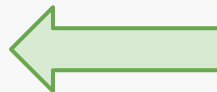
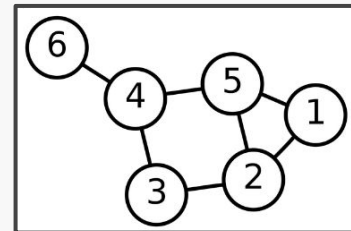
Imágenes



CNNs, Vision Transformers, ...

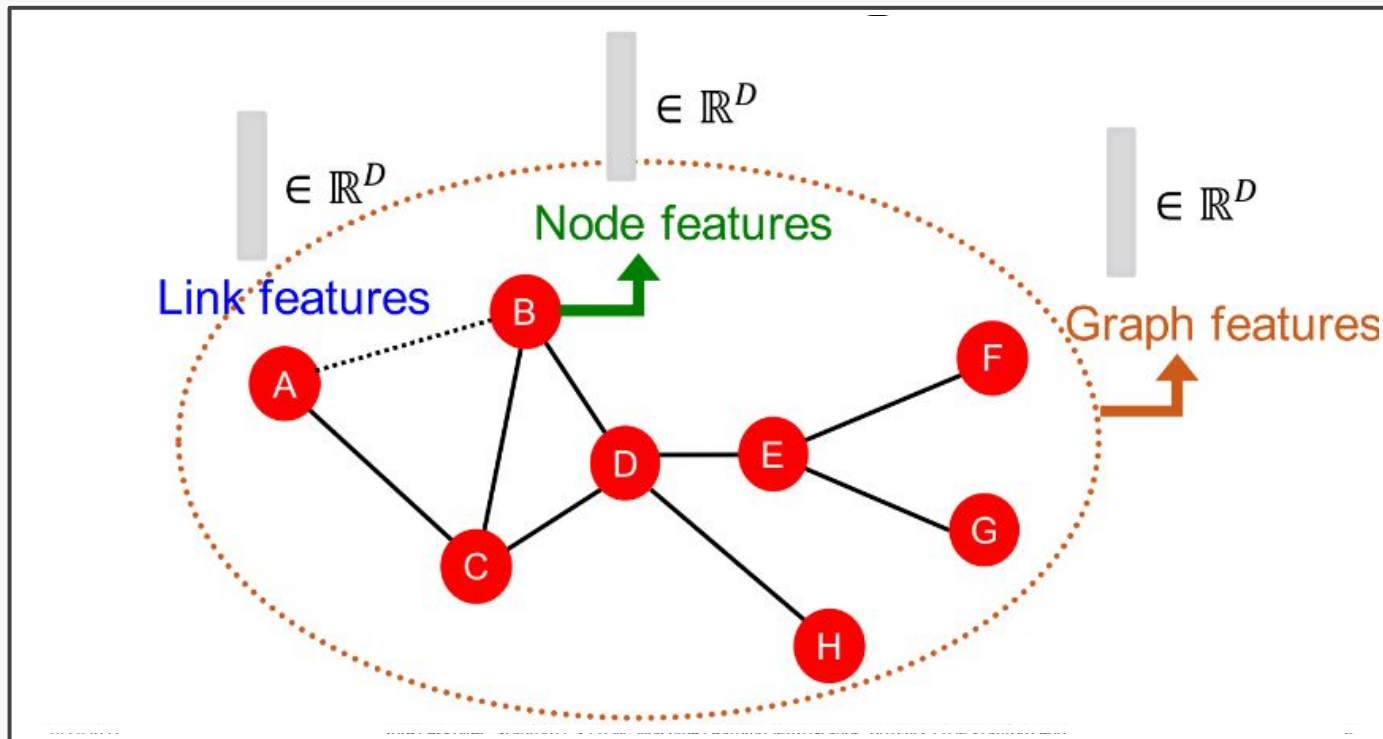


Grafos



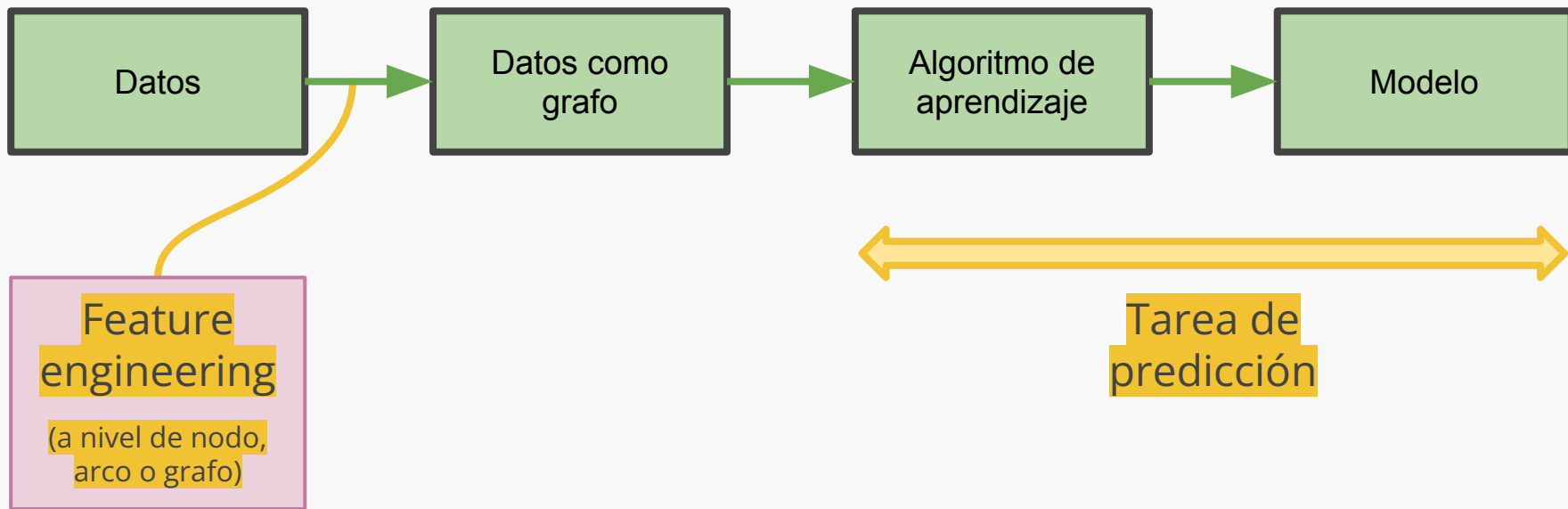
???

ML tradicional: necesitamos features



Como en todo problema de Machine Learning...

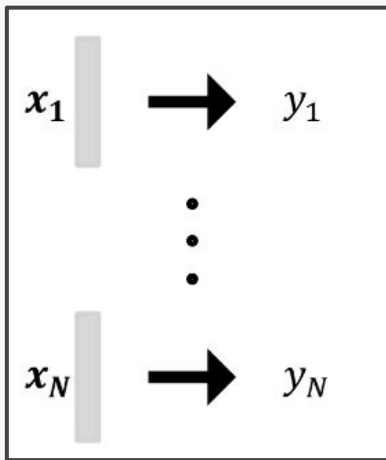
Dadas las *features* de un grafo (o porción), aprender un modelo que convierte las *features* en la salida esperada:



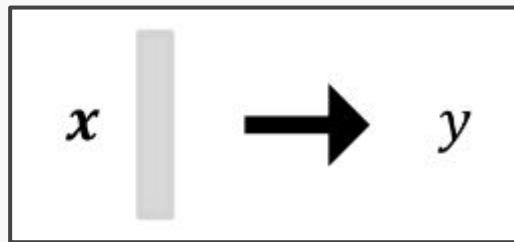
ML tradicional: flujo de entrenamiento

- Entrenamos un modelo:

- Random forest
- SVM
- NN
- etc.



- Aplicamos el modelo:
Dado un nuevo nodo/arco/grafó, obtenemos sus features y hacemos una predicción



Diseño de features desde analítica de grafos

Objetivo: hacer predicciones para un conjunto de objetos

Decisiones de diseño:

- **Features:** vectores multidimensionales
- **Objetos:** nodos, arcos, conjuntos de nodos, grafos enteros, etc.
- **Función objetivo:**
 - ¿Cuál es la tarea que queremos resolver?

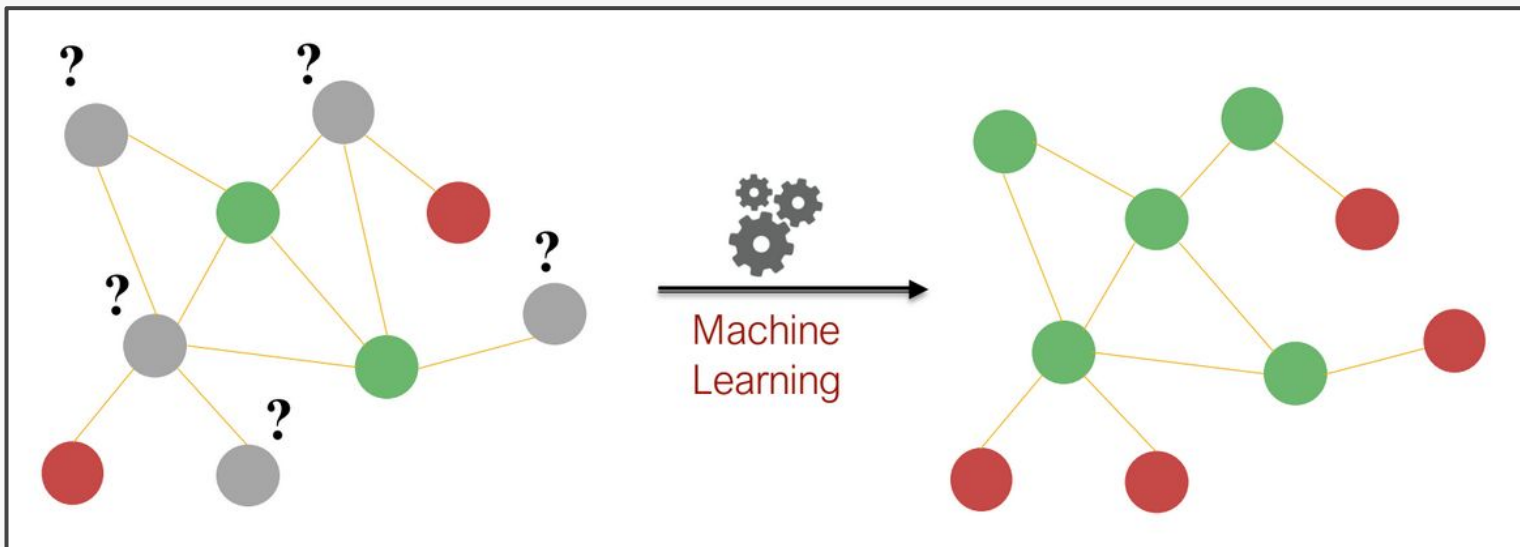
Agenda

✨ **Métricas para nodos**

Métricas para arcos

Mátricas para grafos

Predicción a nivel de nodo

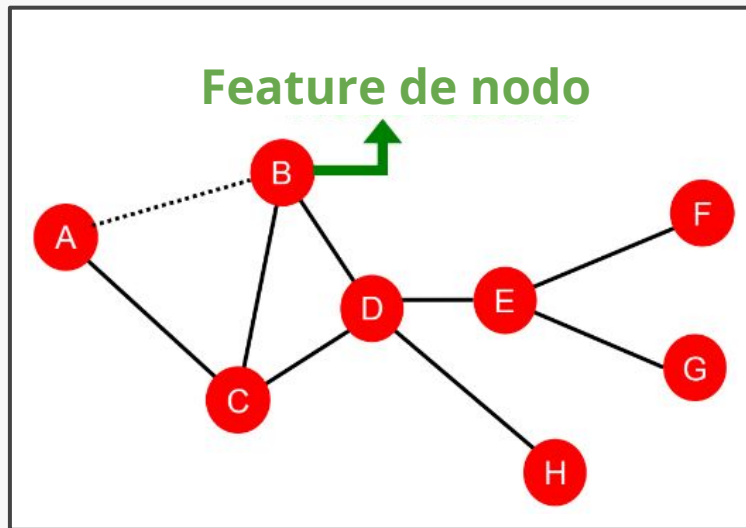


Dado un grafo $\mathbf{G} = (\mathbf{V}, \mathbf{E})$, queremos aprender una función $\mathbf{f}: \mathbf{V} \rightarrow \mathbf{R}$
¿Cómo podemos aprender tal función?

Features de nodos

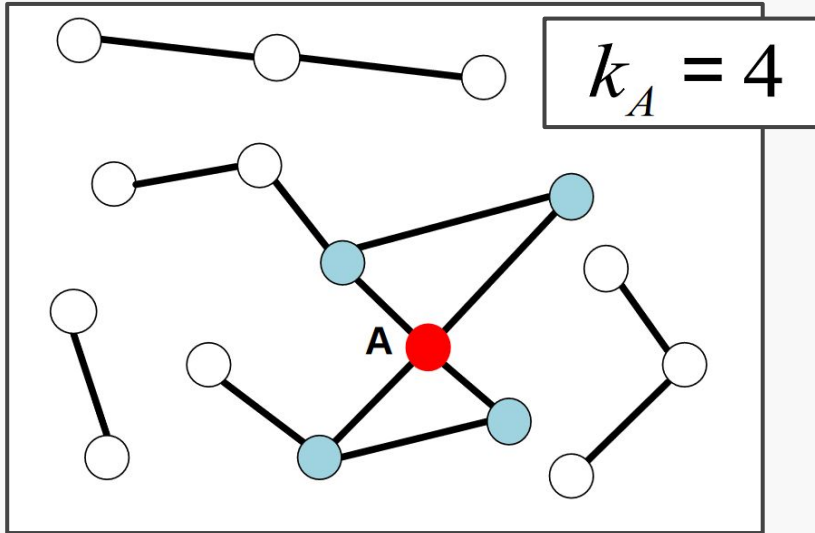
Objetivo: Caracterizar la estructura y posición de un nodo en una red:

- Grado de un nodo
- Centralidad de un nodo
- Coeficiente de clustering
- Graphlets



Grado de un nodo

El **grado de un vértice/nodo** es el número de aristas incidentes al vértice, básicamente, el **número de arcos que se conectan a un nodo particular**:

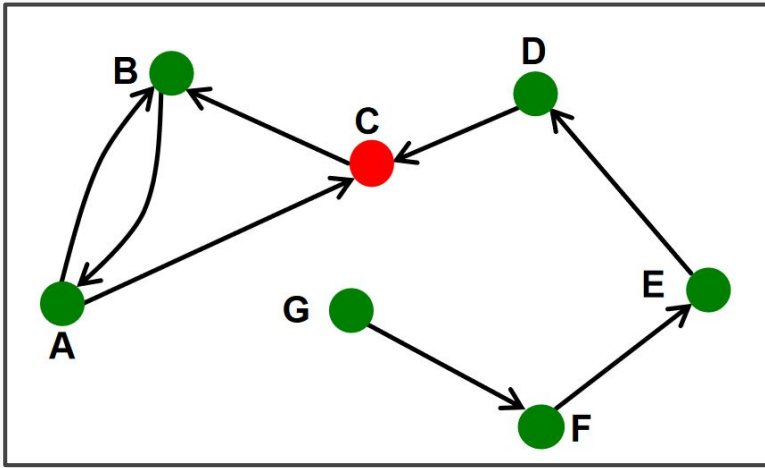


Grado promedio:

$$\bar{k} = \langle k \rangle = \frac{1}{N} \sum_{i=1}^N k_i = \frac{2E}{N}$$

Grado de un nodo

En el caso de grafos dirigidos, definimos un **grado de entrada** (in-degree) y un **grado de salida** (out-degree) según la cantidad de arcos que llegan o parten desde el nodo de interés



$$k_C^{in} = 2 \quad k_C^{out} = 1 \quad k_C = 3$$

$$\bar{k} = \frac{E}{N}$$

Grado
promedio

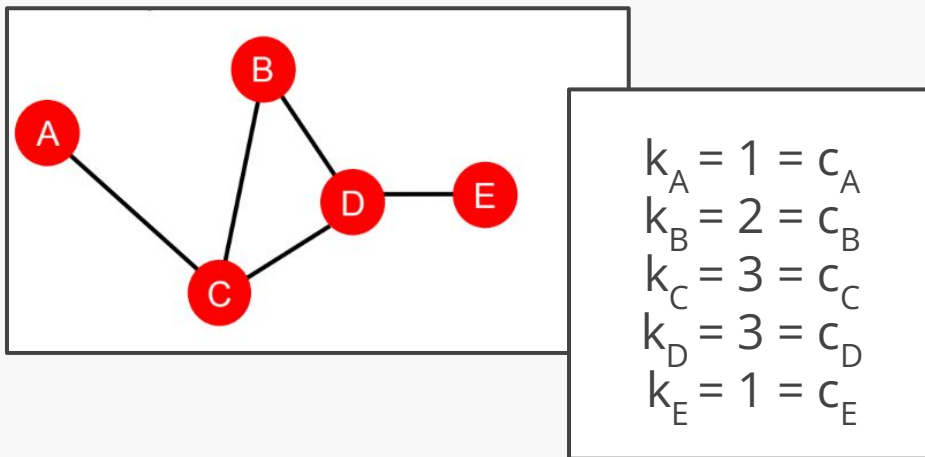
$$\overline{k^{in}} = \overline{k^{out}}$$

Centralidad de un nodo

- El grado de un nodo cuenta los nodos vecinos, pero no toma en cuenta su importancia
- La centralidad de un nodo (c_v) toma en cuenta la importancia de un nodo en un grafo
- Existen diferentes maneras de modelar importancia
 - Centralidad de vector propio (*eigenvector centrality*)
 - Centralidad de intermediación (*betweenness centrality*)
 - Centralidad de cercanía (*closeness centrality*)
 - Otras maneras adicionales

Cent. de grado (*degree centrality*)

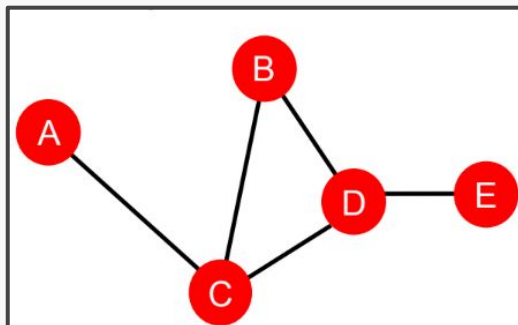
- La medida más simple, cuenta cuántos arcos tiene cada nodo: el nodo más “central” es el que tiene más arcos



Cent. de intermediación (*betweenness centrality*)

- Un nodo es importante si se ubica en muchas “rutas más cortas” entre otros pares de nodos:

$$c_v = \sum_{s \neq v \neq t} \frac{\#(\text{rutas mínimas entre } s \text{ y } t \text{ que contienen a } v)}{\#(\text{rutas mínimas entre } s \text{ y } t)}$$



$$c_A = c_B = c_E = 0$$
$$c_C = 3$$

(A-C-B, A-C-D, A-C-D-E)

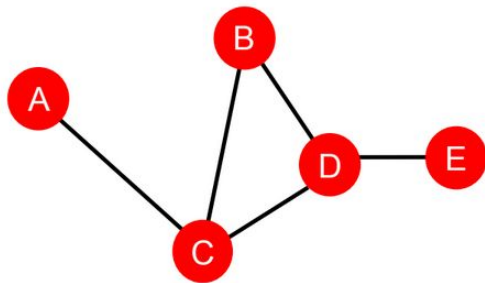
$$c_D = 3$$

(A-C-D-E, B-D-E, C-D-E)

Cent. de cercanía (*closeness centrality*)

- Un nodo es importante si tiene una pequeña “distancia más corta” a todos los otros nodos del grafo:

$$c_v = \frac{1}{\sum_{u \neq v} \text{largo de ruta mínima entre } u \text{ y } v}$$



$$c_A = 1/(2 + 1 + 2 + 3) = 1/8$$

(A-C-B, A-C, A-C-D, A-C-D-E)

$$c_D = 1/(2 + 1 + 1 + 1) = 1/5$$

(D-C-A, D-B, D-C, D-E)

Cent. de vector propio (*eigenvector centrality*)

- Un nodo v es importante si está rodeado de nodos importantes $u \in N(v)$
- Modelamos la centralidad del nodo v como la suma de la importancia de centralidad de los nodos vecinos:

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u$$

- Notar que esta ecuación modela la centralidad de manera recursiva

¿Cómo podemos resolver esta ecuación?

Cent. de vector propio (*eigenvector centrality*)

- Reescribimos la ecuación recursiva en su forma matricial:

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u$$



$$\lambda \mathbf{c} = \mathbf{A} \mathbf{c}$$

λ es const. de normalización
(máximo valor propio de A)

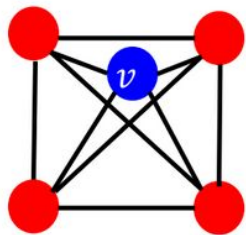
A : matriz de adyacencia
 $\mathbf{c}$: vector de centralidad
 λ : valor propio

- Vemos que la centralidad $\mathbf{c}$ es el vector propio de A
- El valor propio más alto siempre es positivo y único (teorema de Perron-Frobenius)
- El vector propio $\mathbf{c}_{\max}$ correspondiente a $\lambda_{\max}$ es usado para la centralidad

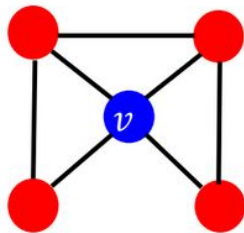
Coeficiente de *clustering*

- Mide qué tan conectados son los vecinos del nodo v :

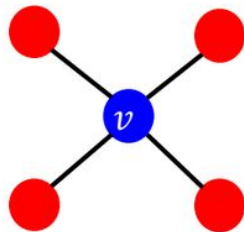
$$e_v = \frac{\#(\text{arcos entre nodos que son vecinos})}{\binom{k_v}{2} \#(\text{pares de nodos entre } k_v \text{ nodos vecinos})} \in [0,1]$$



$$e_v = 1$$



$$e_v = 0.5$$



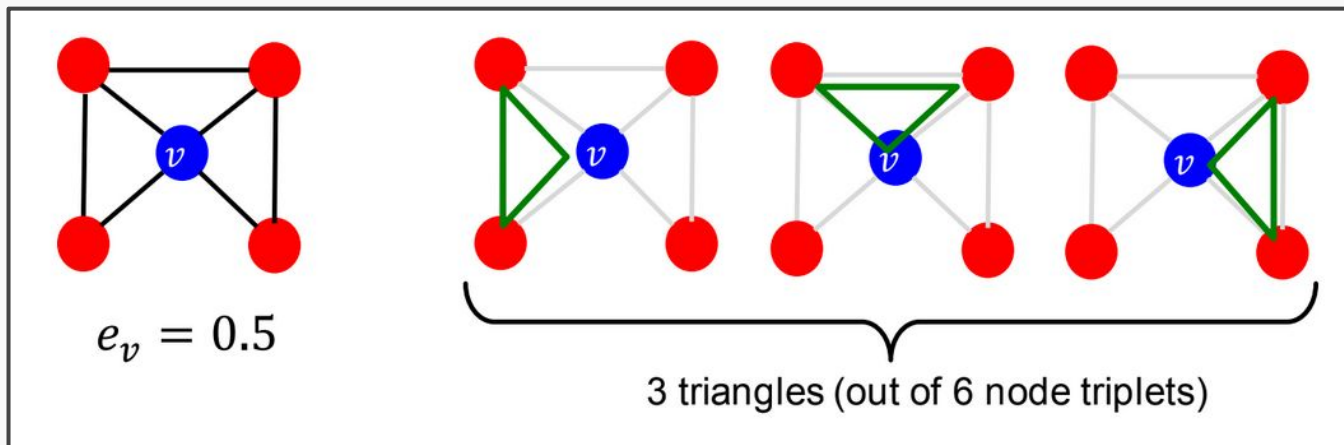
$$e_v = 0$$

En este ejemplo, el denominador es 6: tenemos una combinatoria de 4 sobre 2, es decir:

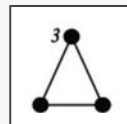
$$4! / (2! \times (4 - 2)!)$$

Antes de graphlets

Observación: El coeficiente de clustering cuenta el número de triángulos



- Podemos generalizar esto contando el número de veces que aparecen sub-grafos específicos (graphlets):



Graphlets

Objetivo: describir la estructura de la red alrededor del nodo v

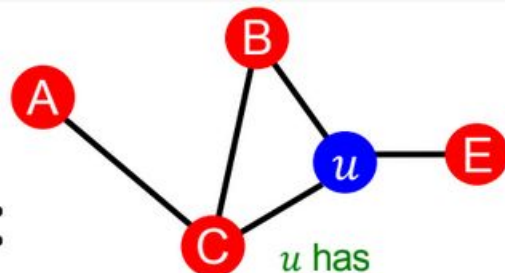
- Graphlets son pequeños sub-grafos que describen la estructura del grafo vecindario al nodo v

Analogía:

- El **grado** cuenta el número de arcos que un nodo toca
- El **coeficiente de clustering** cuenta los triángulos que un nodo toca
- El **Graphlet Degree Vector (GDC)** describe features basadas en graphlets
 - GDV cuenta el número de graphlets que un nodo toca

Graphlets: Rooted connected induced non-isomorphic subgraphs:

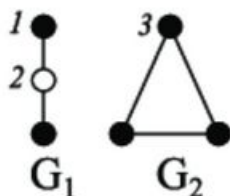
Take some nodes and all the edges between them.



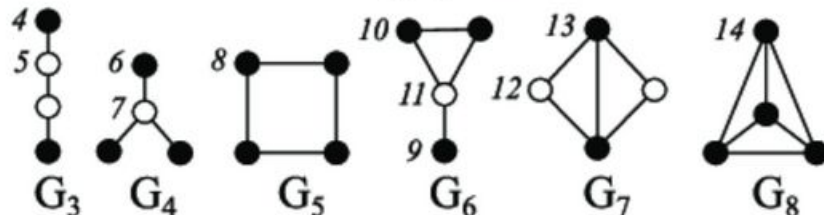
2-node graphlet



3-node graphlets

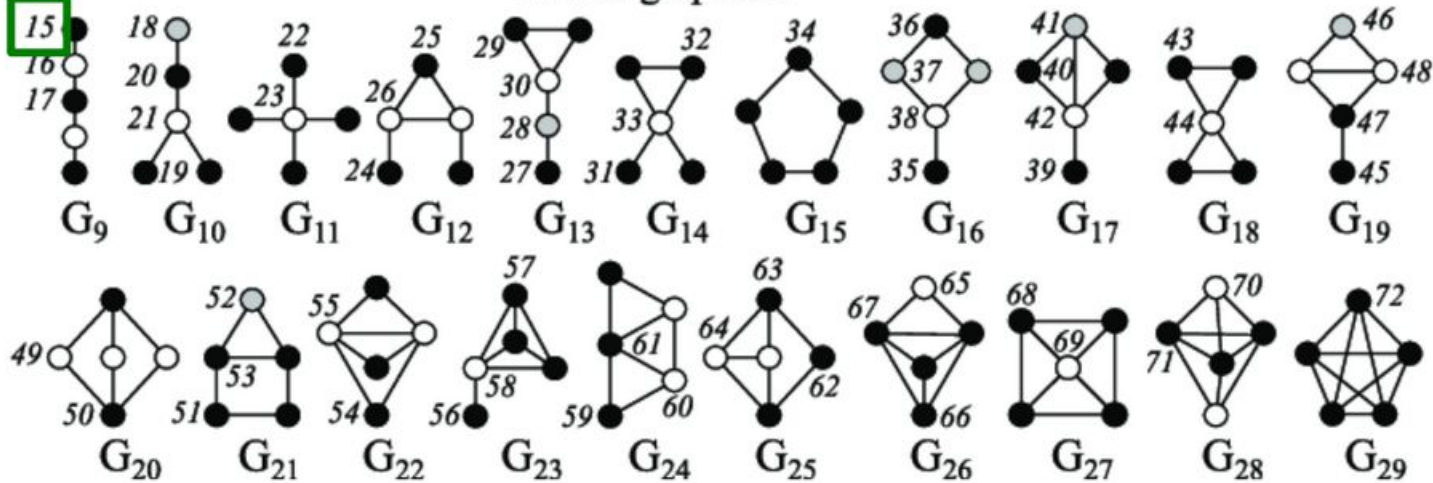


4-node graphlets



Graphlet id (Root/ "position" of node u)

5-node graphlets

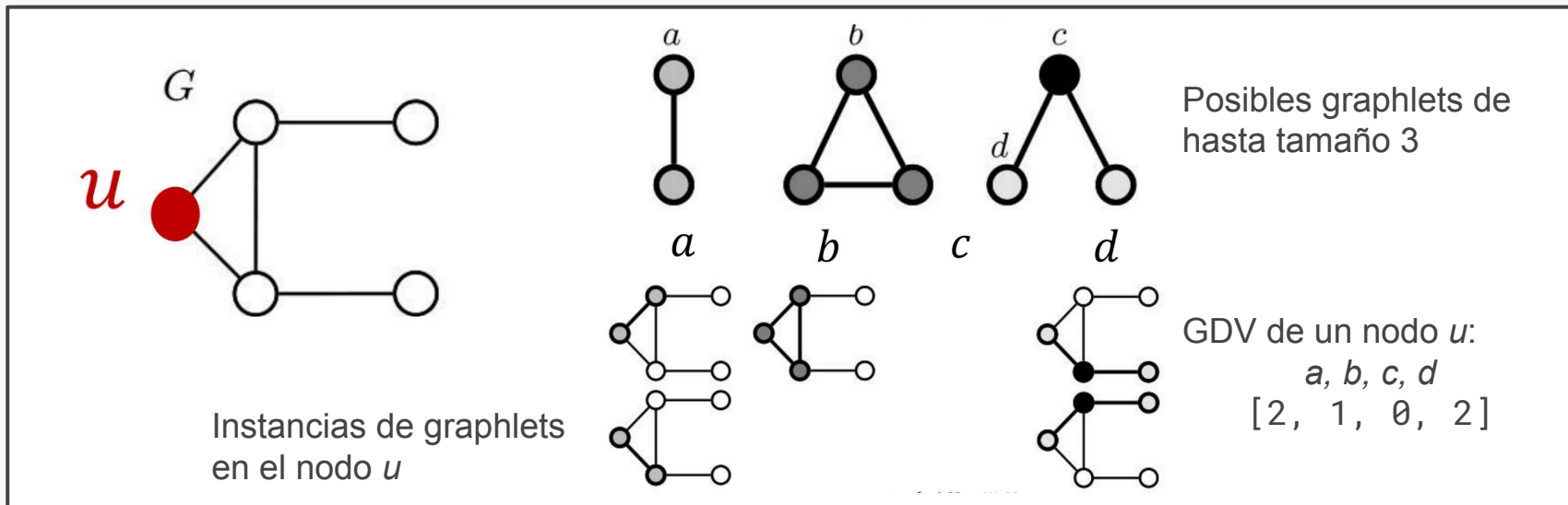


Graphlets

- Al considerar los graphlets de entre 2 y 5 nodos, obtenemos un **vector de 73 coordenadas** que podrá ser usado como firma (*signature*) que describe la topología del vecindario de un nodo
- Este vector entrega una medida de **la topología de la vecindad local de un nodo**:
 - Comparar el vector de dos nodos entrega una medida más detallada de la **similitud de la topología local** que utilizar los grados de los nodos (solo nos dice el número de vecinos, pero no cómo se relacionan los vecinos) o utilizar el coeficiente de clustering (nos dice qué tan conectados están los vecinos, pero no cómo es esa topología)

Graphlets

- **Graphlet Degree Vector (GDV):** Un vector que cuenta los graphlets que ocurren en un nodo (entre un nodo y su vecindad)
- Ejemplo:



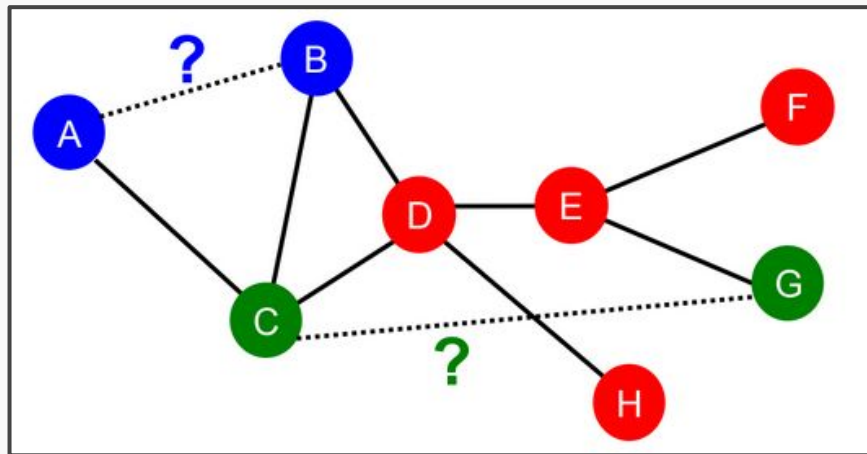
Agenda

Métricas para nodos

✨ **Métricas para arcos**

Mátricas para grafos

Predicción a nivel de arco



- Queremos predecir nuevos arcos basándonos en los arcos existentes
- Al predecir, tomamos los pares de nodos sin arco, los ordenamos y predecimos los primeros K
- La clave es diseñar features para un par de nodos

Predicción a nivel de arco

Tenemos dos formulaciones posibles:

1) Arcos faltantes al azar (*links missing at random*)

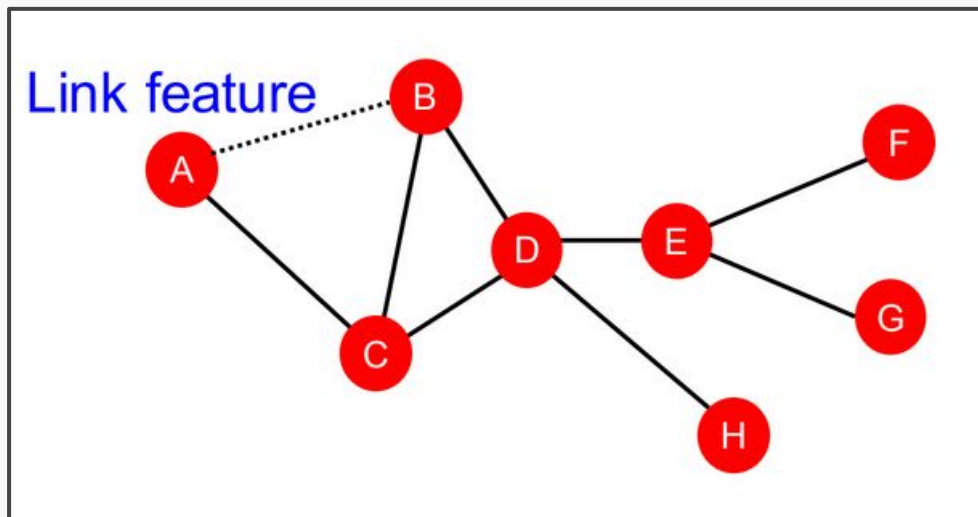
- a) Quitamos un subconjunto de arcos al azar
- b) Intentamos predecir los arcos que quitamos

2) Arcos en el tiempo (*links over time*)

- a) Dado un grafo definido por los arcos hasta un tiempo t
- b) Retornar una lista ordenada de arcos que creemos aparecerán en $t+1$
- c) Si sabemos que aparecen N arcos entre t y $t+1$, tomamos los N arcos más “probables” en la predicción

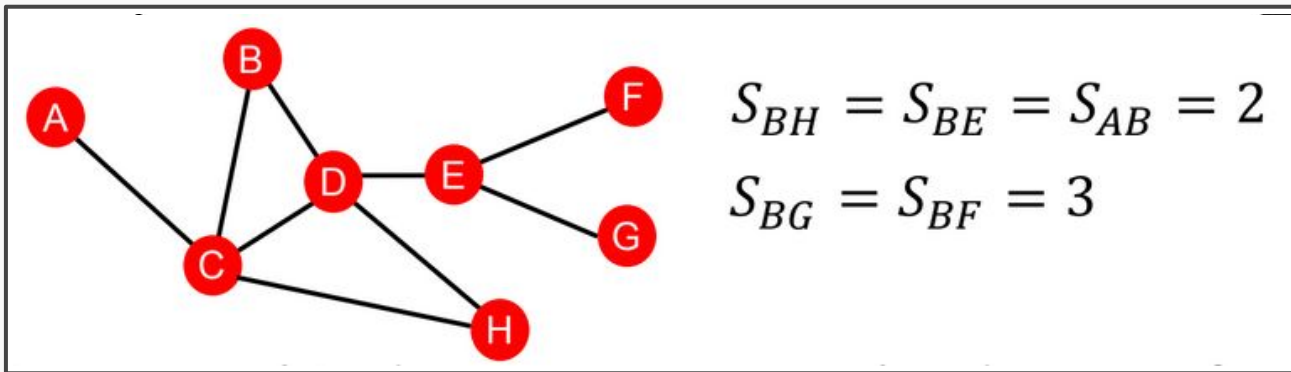
Features de arcos

- Features basadas en distancia
- Superposición en vecindario local (*local neighborhood overlap*)
- Superposición en vecindario global (*global neighborhood overlap*)



Features basadas en distancia

- Camino más corto entre dos nodos:



- Sin embargo, no captura el grado de superposición de vecindarios:
 - El par de nodos (B, H) tiene dos vecinos en común, mientras que los pares (B, E) y (A, B) solo tienen un nodo en común

Local neighborhood overlap

Captura el número de nodos vecinos compartidos entre dos nodos v_1 y v_2 :

- Vecinos en común: $|N(v_1) \cap N(v_2)|$

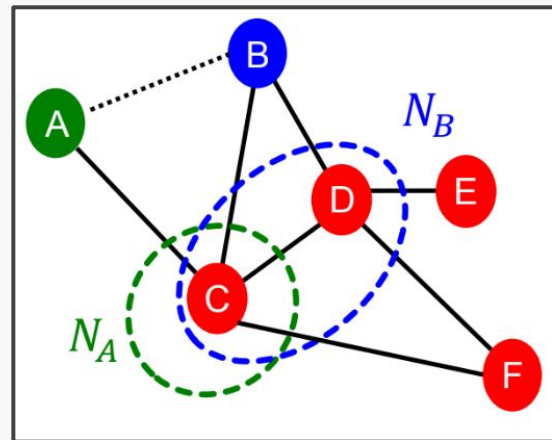
- Ejemplo: $|N(A) \cap N(B)| = |\{C\}| = 1$

- Coeficiente de Jaccard: $\frac{|N(v_1) \cap N(v_2)|}{|N(v_1) \cup N(v_2)|}$

- Ejemplo: $\frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|} = \frac{|\{C\}|}{|\{C, D\}|} = \frac{1}{2}$

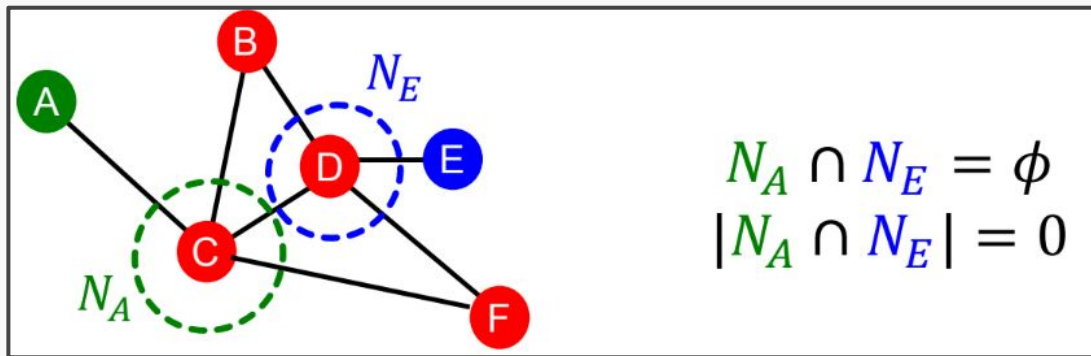
- Índice Adamic-Adar: $\sum_{u \in N(v_1) \cap N(v_2)} \frac{1}{\log(k_u)}$

- Ejemplo: $\frac{1}{\log(k_C)} = \frac{1}{\log 4}$



Global neighborhood overlap

- Limitación de *local neighborhood overlap*:
 - La métrica siempre es cero si los nodos no tienen vecinos en común



- Sin embargo, los nodos podrían estar potencialmente conectados en el futuro
- Métricas de *Global neighborhood overlap* resuelven esta limitación considerando el grafo entero

Global neighborhood overlap

- **Índice Katz**: cuenta el número de caminos, de todos los largos, existentes entre un par de nodos

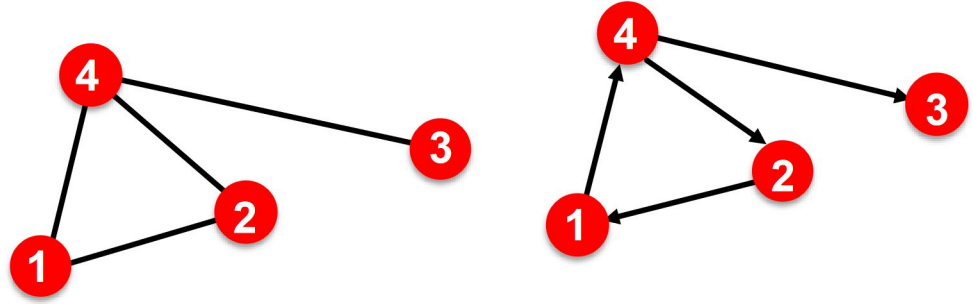
¿Cómo podemos calcular el número de caminos entre dos nodos?

Usando potencias de la matriz de adyacencia

Matriz de adyacencia

Dado un grafo, podemos representarlo con una matriz indicando la existencia de arcos entre nodos

$A_{ij} = 1$ si existe un arco de i a j
 $A_{ij} = 0$ en caso contrario



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Potencia de una matriz de adyacencia

El número de caminos que pasa por k aristas (camino de largo k) desde el nodo i al nodo j , es el elemento en la fila i y columna j de la matriz de adyacencia elevada a k (A^k)

- 1) Calculamos el número de caminos de largo 1 entre cada vecino de u y v
- 2) Sumar todos estos caminos a lo largo de los vecinos de u

$$\blacksquare P_{uv}^{(2)} = \sum_i A_{ui} * P_{iv}^{(1)} = \sum_i A_{ui} * A_{iv} = A_{uv}^2$$

Node 1's neighbors #walks of length 1 between Node 1's neighbors and Node 2 $P_{12}^{(2)} = A_{12}^2$

$$A^2 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \times \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}$$

Power of adjacency

Global neighborhood overlap

- **Índice Katz:** cuenta el número de caminos, de todos los largos, existentes entre un par de nodos

Potencias de la matriz de adyacencia:

- A_{uv} especifica el # de caminos de largo 1 entre u y v
- A_{uv}^2 especifica el # de caminos de largo 2 (vecino de vecino) entre u y v
- De la misma manera, A_{uv}^k especifica # de caminos de largo k entre u y v

Global neighborhood overlap

- **Índice Katz** entre v_1 y v_2 es calculado como una suma en todos los largos posibles de camino:

$$S_{v_1 v_2} = \sum_{l=1}^{\infty} \boxed{\beta^l} \boxed{A_{v_1 v_2}^l} \quad \begin{array}{l} \text{\#walks of length } l \\ \text{between } v_1 \text{ and } v_2 \end{array}$$

$0 < \beta < 1$: discount factor

Beta es un peso de descuento para los vecindarios más lejanos, haciendo que tengan menos peso si están más lejos de v_1 y v_2

Agenda

Métricas para nodos

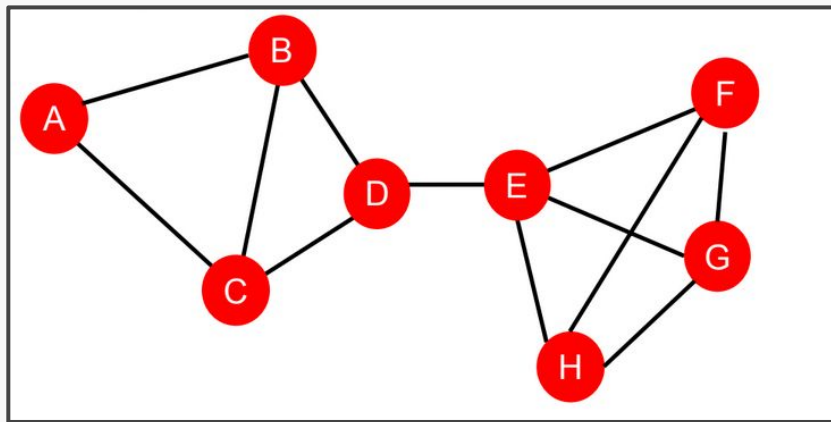
Métricas para arcos

✨ **Métricas para grafos**

Features de grafos

Objetivo: Queremos features que describen la estructura completa de un grafo

- Kernel de graphlets
- Kernel de Weisfeiler-Lehman



Kernel: Función de similaridad entre grafos

Métodos kernel

Idea: Diseñar kernels en lugar de vectores de *features*

Introducción a kernels:

- Kernel $K(G, G') \in \mathbb{R}$ mide la similaridad entre grafos
- La matriz kernel $\mathbf{K} = (K(G, G'))_{G, G'}$, siempre debe ser positiva semidefinida (es decir, debe tener valores propios positivos)
- Existe una representación de features $\phi(\cdot)$ tal que $K(G, G') = \phi(G)^T \phi(G')$
- Una vez que la kernel está definida, modelos como SVM pueden usarse para hacer predicciones

Kernels para grafos

¿Por qué kernels? El objetivo es medir similaridad entre grafos para poder compararlos y con ello diferenciarlos

- Kernel de graphlets
- Kernel de Weisfeiler-Lehman (Kernel WL)
- (Hay más kernels en la literatura)

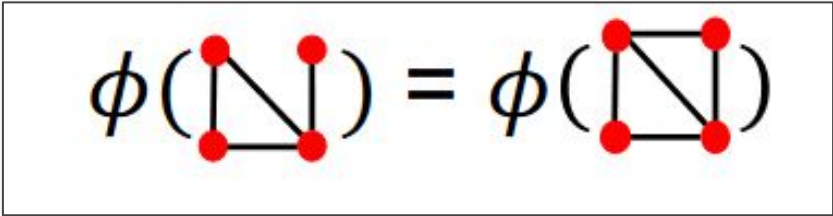
Kernels para grafos: idea clave

Objetivo: Diseñar un vector de features $\phi(G)$

Idea clave: Bag-of-words (BoW) pero para grafos

- BoW usa el número de palabras en un documento para construir features (no se considera el orden). **Primera idea: tratar a los nodos como palabras**

Ejemplo: estos grafos tienen 4 nodos, podemos decir que ambos grafos tienen el mismo vector de features... a pesar de que sean distintos grafos


$$\phi(\text{graph 1}) = \phi(\text{graph 2})$$

Kernels para grafos: idea clave

¿Qué tal usar una “Bag-of-(grados de nodos)”?

Deg1: ● Deg2: ● Deg3: ●

$$\phi(\text{Graph 1}) = \text{count}(\text{Graph 2}) = [1, 2, 1]$$

$$\phi(\text{Graph 3}) = \text{count}(\text{Graph 4}) = [0, 2, 2]$$



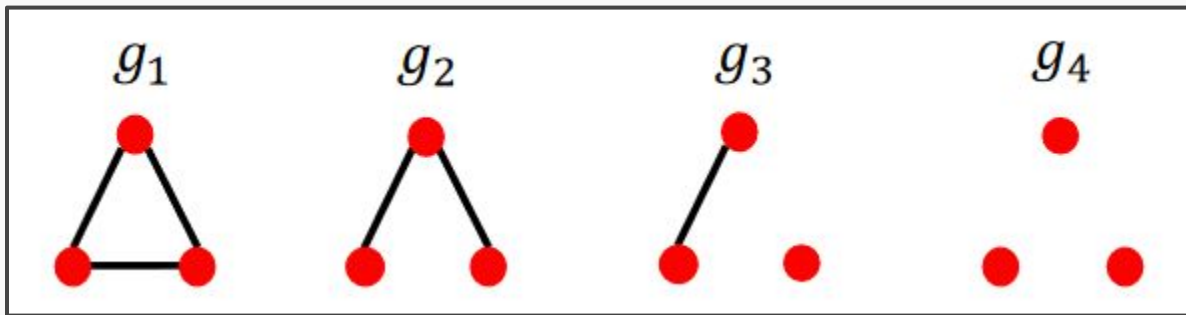
Obtains different features
for different graphs!

Tanto el kernel de graphlets como el kernel Weisfeiler-Lehman usan una representación a partir de la idea de **Bag-of-***, donde * es más sofisticado que el grado de un nodo

Features de graphlets a nivel de grafo

Idea clave: Contar el número de graphlets diferentes en un grafo

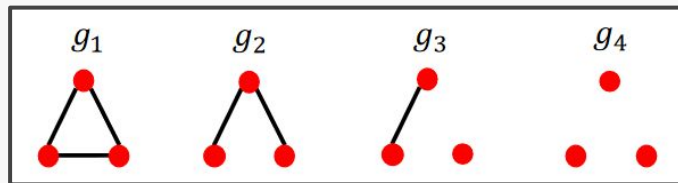
- **Nota:** La definición de graphlets es levemente diferente de aquella que vimos en las features a nivel de nodos. Las dos diferencias son:
 - Los nodos en los graphlets no necesitan estar conectados (esto permite usar nodos aislados)
 - Los graphlets no tienen una raíz



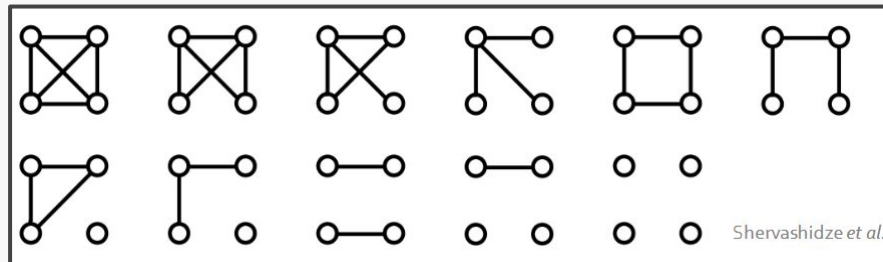
Features de graphlets a nivel de grafo

Sea $G_k = (g_1, g_2, \dots, g_{nk})$ una lista de graphlets de tamaño k :

- Para $k = 3$ existen 4 graphlets:



- Para $k = 4$ existen 11 graphlets:



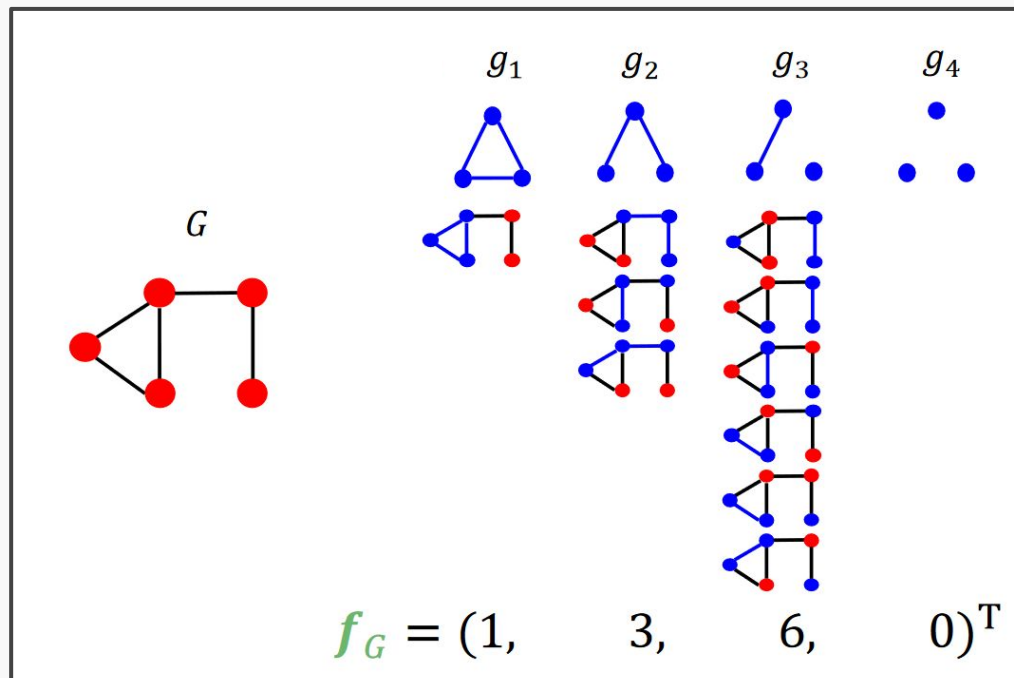
Features de graphlets a nivel de grafo

Dado un grafo G y una lista de graphlets $G_k = (g_1, g_2, \dots, g_{n_k})$, definimos el vector de conteo de graphlets $f_G \in R^{n_k}$ como

$$(f_G)_i = \#(g_i \subseteq G) \text{ for } i = 1, 2, \dots, n_k$$

Features de graphlets a nivel de grafo

Por ejemplo, para $k = 3$:



Kernel de graphlets a nivel de grafo

Dado dos grafos, G y G' , el kernel de graphlets se calcula como:

$$K(G, G') = \mathbf{f}_G^T \mathbf{f}_{G'}$$

Problema: si G y G' tienen tamaños distintos, eso sesgará los valores

Solución: normalizar cada vector de features

$$\mathbf{h}_G = \frac{\mathbf{f}_G}{\text{Sum}(\mathbf{f}_G)} \quad K(G, G') = \mathbf{h}_G^T \mathbf{h}_{G'}$$

Kernel de graphlets a nivel de grafo

Limitaciones: Contar graphlets es caro

- Contar graphlets de tamaño k para un grafo de tamaño n toma n^k
- Esto es inevitable en el peor caso debido a que la prueba de isomorfismo de subgrafos (juzgar si un grafo es subgrafo de otro grafo) es un problema NP-hard
- Si el grado del nodo de un grafo está acotado a d , existe un algoritmo $O(nd^{k-1})$ que cuente todos los graphlets de tamaño k

¿Podemos diseñar un kernel de grafos más eficiente?

Kernel Weisfeiler-Lehman (Kernel WL)

Objetivo: Diseñar un descriptor de features de grafo $\phi(G)$ que sea eficiente

Idea: Usar la estructura de vecindario para enriquecer el vocabulario de un nodo

- Versión generalizada de la idea de “**Bag-of-(grado de un nodo)**” ya que los grados de un nodo entregan información del vecindario a distancia 1

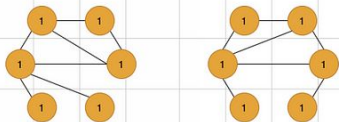
Algoritmo a usar:

Coloración de grafos

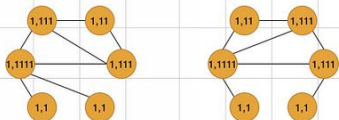
Coloración de grafos

Color Refinement

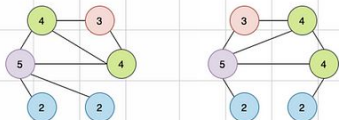
Step 1. 產生初始顏色



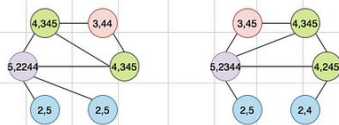
Step 2. Aggregate neighboring color



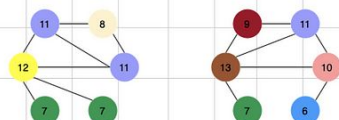
Step 3. 利用 hash map 轉換顏色



Step 4. 再次 aggregate neighboring color



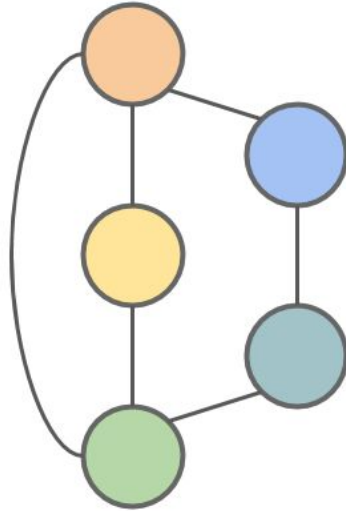
Step 5. 再次利用 Hash map 轉換顏色



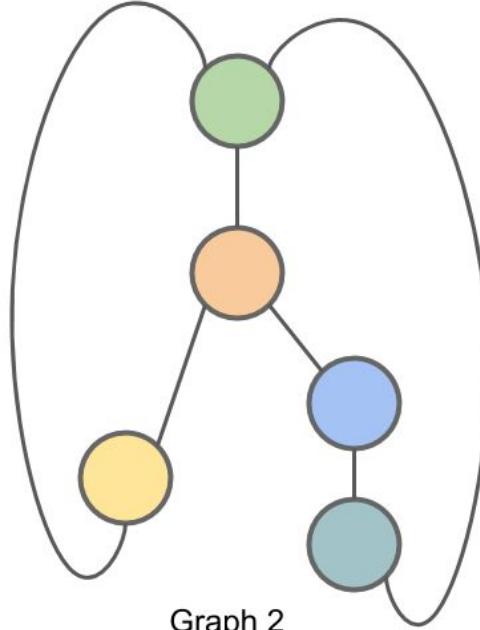
Step 6. 得到 kernel features

Colors = [1,2,3,4,5,6,7,8,9,10,11,12,13]
 $\emptyset(\text{right graph}) = [6,2,1,2,1,0,2,1,0, 0, 0, 2, 1]$
 $\emptyset(\text{left graph}) = [6,2,1,2,1,1,1,0,1, 1, 1, 0, 1]$

Prueba de isomorfismo de Weisfeiler-Lehman



Graph 1

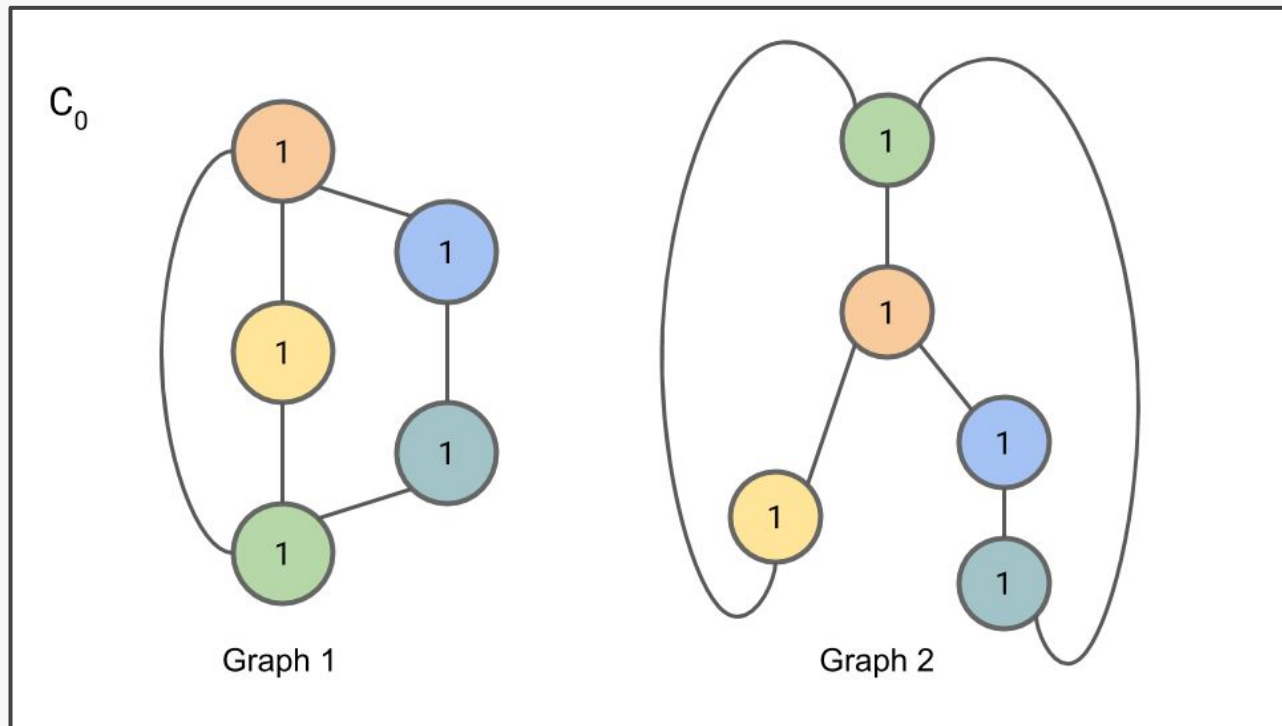


Graph 2

Prueba de isomorfismo de Weisfeiler-Lehman

Iteración 1:

Inicializamos $C_{0,n} = 1$
para todos los nodos
(etiqueta
"comprimida")

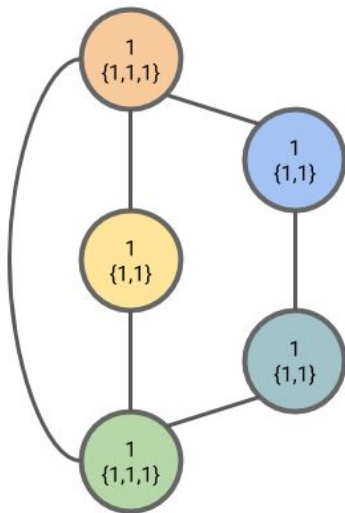


Prueba de isomorfismo de Weisfeiler-Lehman

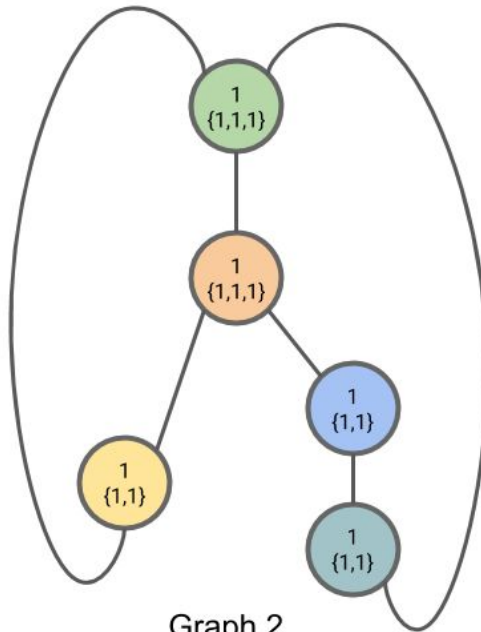
It. 1, Paso 2:

Calculamos $L_{1,n}$. Cada nodo hace seguimiento de su valor anterior, y además recibe un conjunto de los valores vecinos

L_1



Graph 1

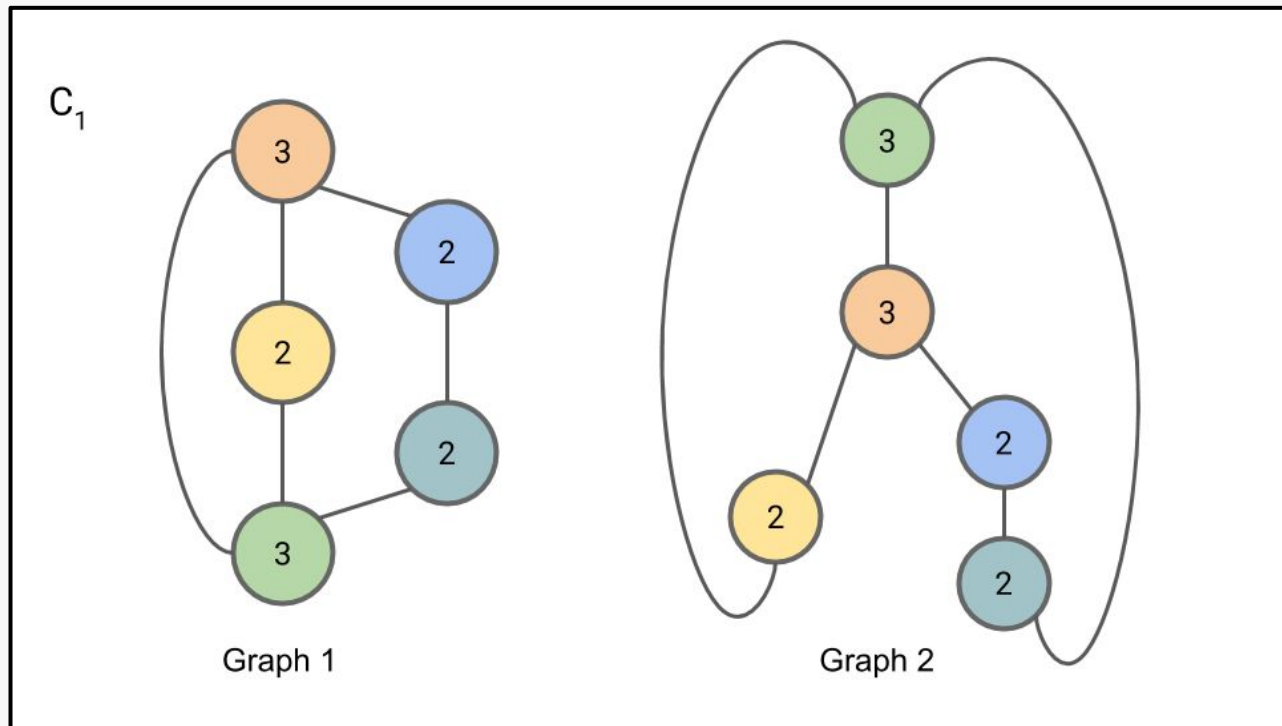


Graph 2

Prueba de isomorfismo de Weisfeiler-Lehman

It. 1, paso 3:

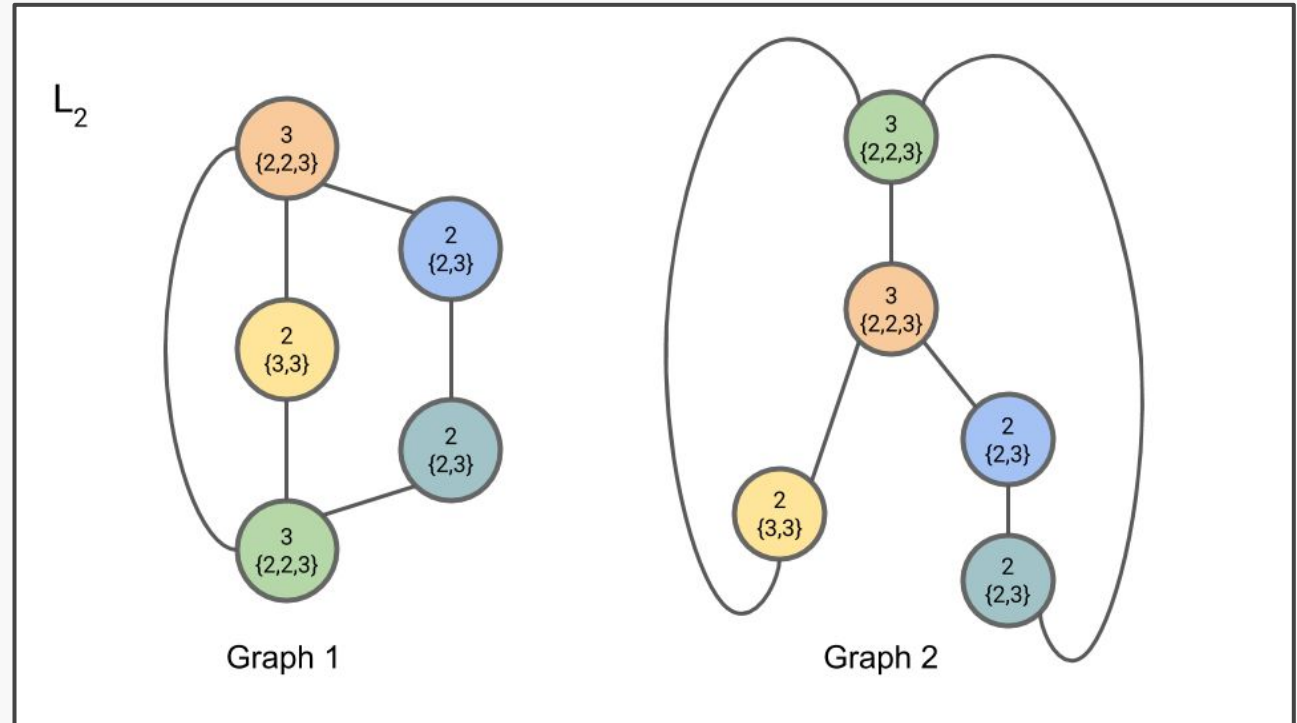
Actualizamos el valor $C_{1,n}$ como la suma de los valores de los nodos vecinos



Prueba de isomorfismo de Weisfeiler-Lehman

It. 2, paso 2:

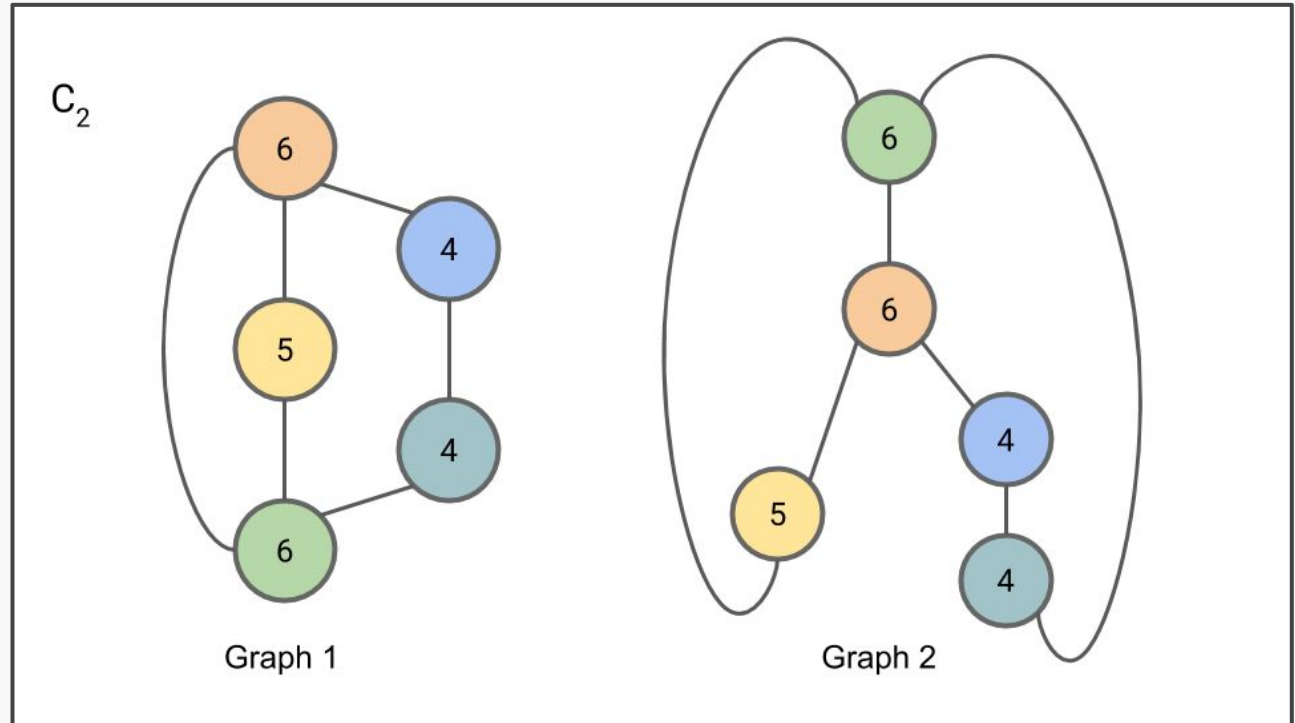
Se calcula $L_{2,n}$



Prueba de isomorfismo de Weisfeiler-Lehman

It. 2, paso 3:

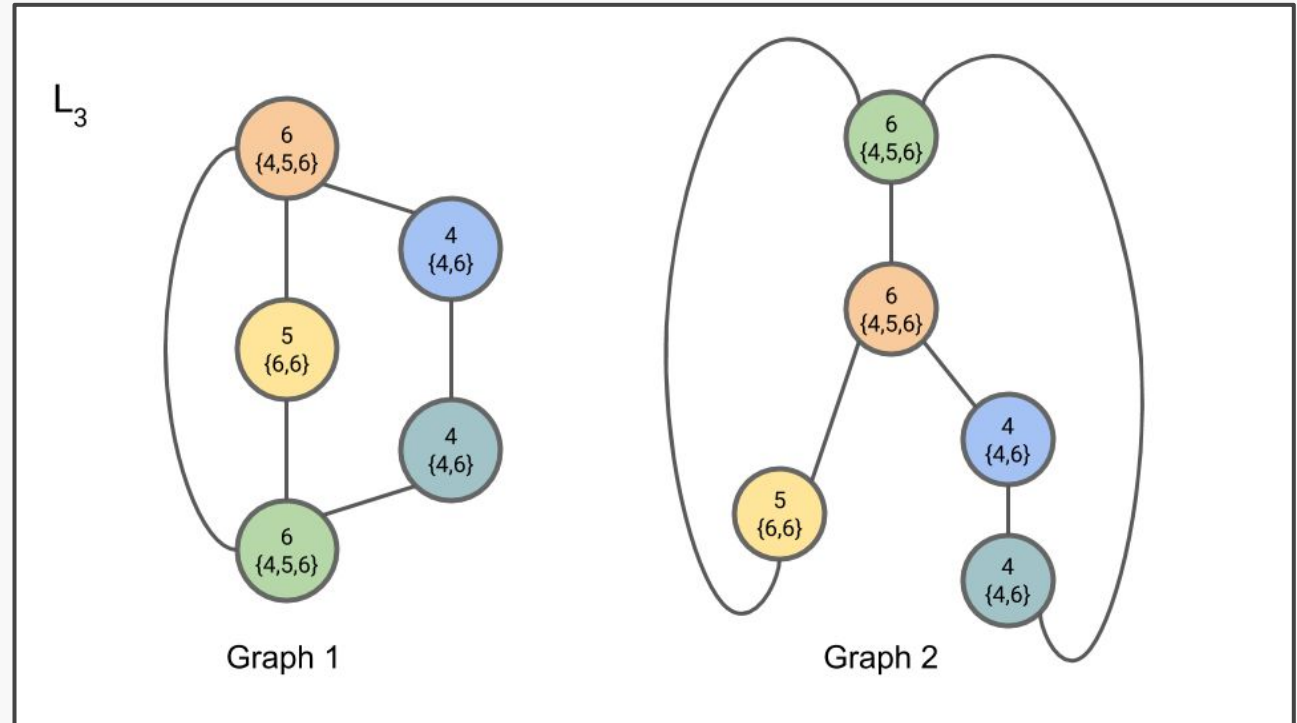
Se calcula $C_{2,n}$ como la suma de las etiquetas vecinas



Prueba de isomorfismo de Weisfeiler-Lehman

It. 3, paso 2:

Calculamos $L_{3,n}$

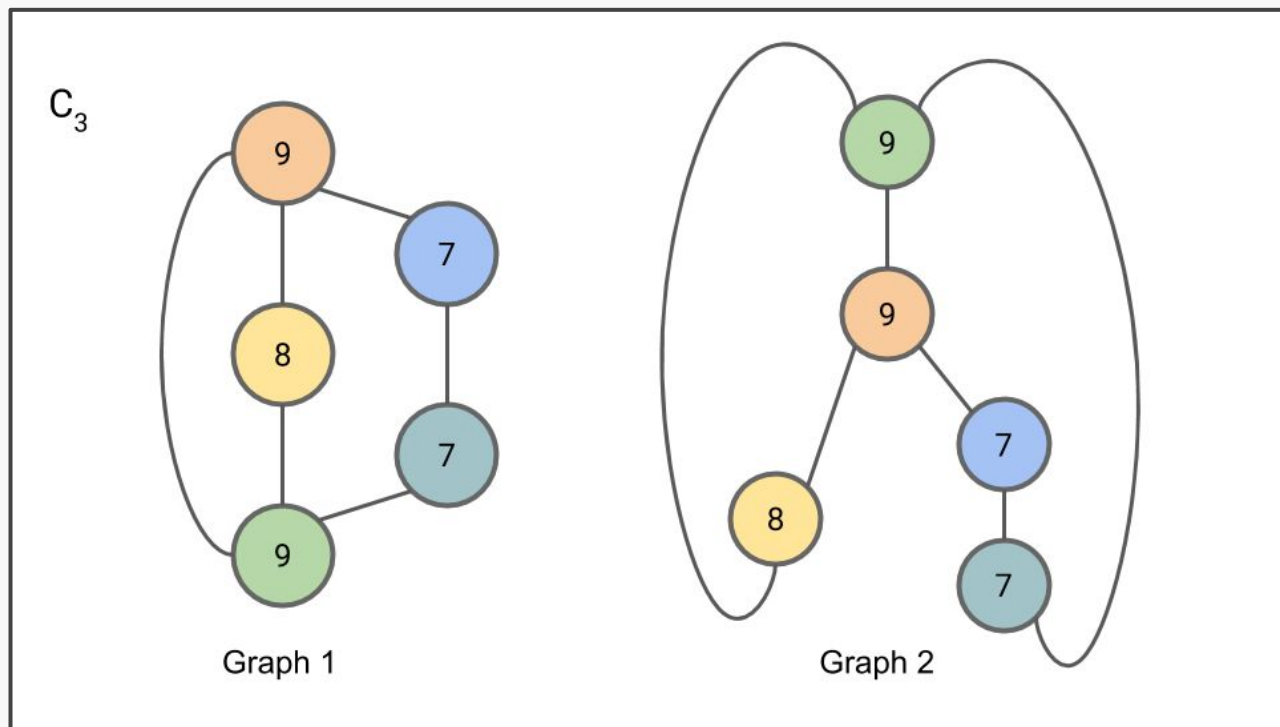


Prueba de isomorfismo de Weisfeiler-Lehman

It. 3, paso 3:

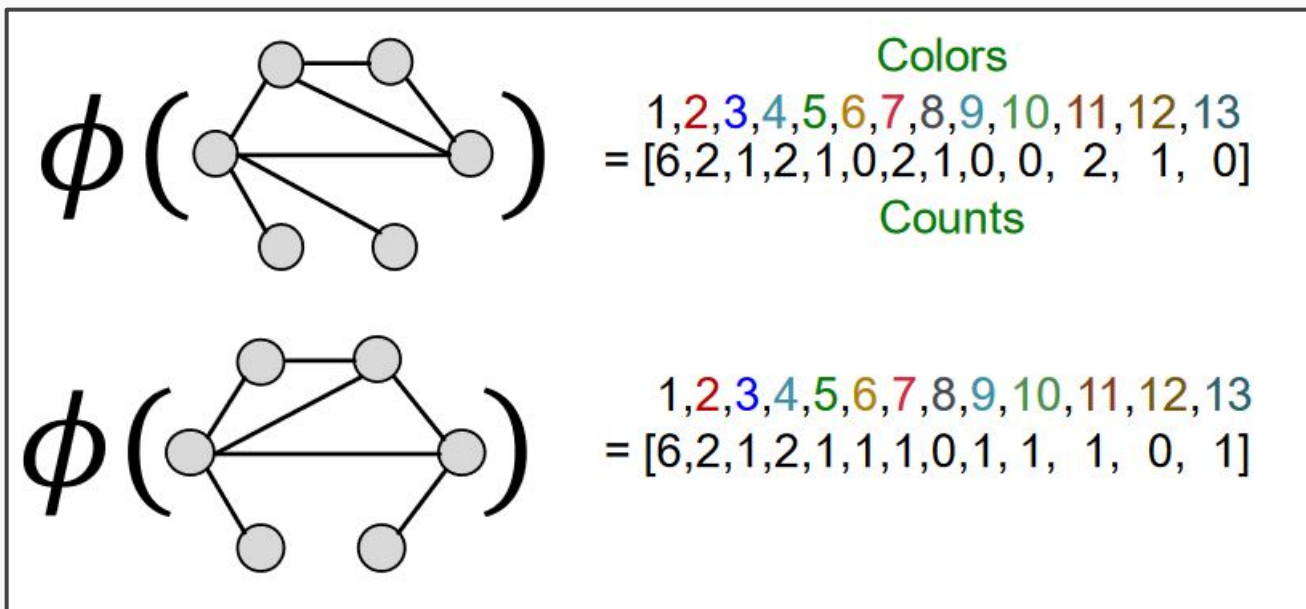
Calculamos $C_{3,n}$.

Dado que la partición de nodos no cambió entre $C_{2,n}$ y $C_{3,n}$, terminamos el algoritmo



Kernel Weisfeiler-Lehman (Kernel WL)

Al terminar, el el kernel WK cuenta los nodos de un color dado y los representa como un vector



Kernel Weisfeiler-Lehman (Kernel WL)

El kernel WL se calcula como el producto interno de los vectores:

$$\begin{aligned} K(\text{Graph}_1, \text{Graph}_2) &= \phi(\text{Graph}_1)^T \phi(\text{Graph}_2) \\ &= 49 \end{aligned}$$

Kernel Weisfeiler-Lehman (Kernel WL)

El Kernel WL es computacionalmente eficiente

- La complejidad computacional del algoritmo en cada paso es lineal (número de nodos), ya que implica agregar los colores de sus vecinos
- Cuando se calcula el valor de un kernel, solo los colores que aparecen en los dos grafos son relevantes
- Contar los colores toma tiempo lineal con respecto al número de nodos
- En total, la complejidad es lineal en el número de nodos

¿Cómo vas con la EP1?



Clase 4.1

Feature engineering

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl